Secrets and Keys Management for Backend-Mediated ARC LLM Access

Technical Whitepaper | August 27, 2026

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

Applications that connect a browser interface to a large language model create a specific credential-management problem. The browser needs enough configuration to select and use a provider, but it must not receive the credential that authorizes provider requests. This whitepaper defines a backend-mediated security model for an application that connects to the Virginia Tech Advanced Research Computing (ARC) OpenAI-compatible LLM service. A FastAPI process reads one personal ARC API key from its environment, validates the provider host, and attaches the authorization header only when sending a TLS-protected request to an approved endpoint. Browser profiles contain non-secret provider settings; routine development and automated testing use mocked providers and require no live credential.

The paper translates that security boundary into operational procedures for Docker Compose, native Windows and POSIX development, testing, repository safeguards, key rotation, and incident response. The central design principle is narrow custody: one credential, one documented environment variable, one backend boundary, and a small set of approved injection paths. The model reduces accidental disclosure without claiming that local environment variables, ignored files, or pattern scanners provide complete secret protection.

1. Purpose and Security Boundary

The project uses one backend-held credential to access an OpenAI-compatible provider. Under the default ARC profile, the credential is the researcher's personal ARC LLM API key. The application names that credential `LLM_API_KEY`; it does not copy the same value into provider-specific aliases or additional variables.

The browser must never request, receive, persist, or display the key. Its provider profile may contain the base URL, model alias, reasoning effort, output limit, timeout, search mode, tool allowlist, and TLS preference because these values are configuration rather than credentials. The FastAPI process reads `LLM_API_KEY` from its environment. It validates the destination against the approved provider-host list before it adds an `Authorization: Bearer <key>` header. The backend then sends the request over TLS to the validated provider.

This boundary separates three concerns:

- The browser controls non-secret interaction and provider preferences.
- The backend controls credential custody, destination validation, and request authorization.
- The provider authenticates the request and executes the selected model operation.

The boundary also defines prohibited disclosure channels. A real credential must not appear in source files, Markdown, prompts, issues, chat transcripts, tests, fixtures, snapshots, logs, telemetry, generated reports, shell commands, screenshots, or browser storage. Team members and coding agents should never ask another person to paste or reveal a key.

2. Secret Inventory and Configuration Classes

The configuration model distinguishes secrets from operational limits and user-facing provider preferences. Treating every configuration value as a secret makes the system harder to operate; treating a credential as ordinary configuration exposes it through normal application and support workflows.

Name or class	Security classification	Purpose	Approved source
`LLM_API_KEY`	Secret	Authorizes provider model, file, and related API requests	Process environment, root `.env` for Compose, or explicitly selected `secrets/local.env`
`ALLOWED_PROVIDER_HOSTS`	Non-secret	Restricts credential-bearing requests to approved hosts	Environment or safe defaults
`MAX_UPSTREAM_OPERATIONS`	Non-secret	Bounds concurrent provider work; default `4`	Environment or safe defaults
`PROVIDER_TIMEOUT_SECONDS`	Non-secret	Sets provider request timeout; default `120` seconds	Environment or safe defaults
`MAX_CONTEXT_CHARACTERS`	Non-secret	Sets local context budget; default `120000` characters	Environment or safe defaults
Browser provider profile	Non-secret	Stores base URL, model, reasoning effort, limits, tools, and TLS preference	Browser IndexedDB
Future MCP bearer tokens and stdio credentials	Secret (future)	Authorizes configured MCP services	Environment variables referenced by configuration, never literal YAML values

`LLM_API_KEY` is the only credential currently required for ARC. Additional ARC secret variables would widen the disclosure surface and create ambiguity during rotation. Future MCP credentials should remain separate because they authorize different services and may have different owners, scopes, and lifetimes.

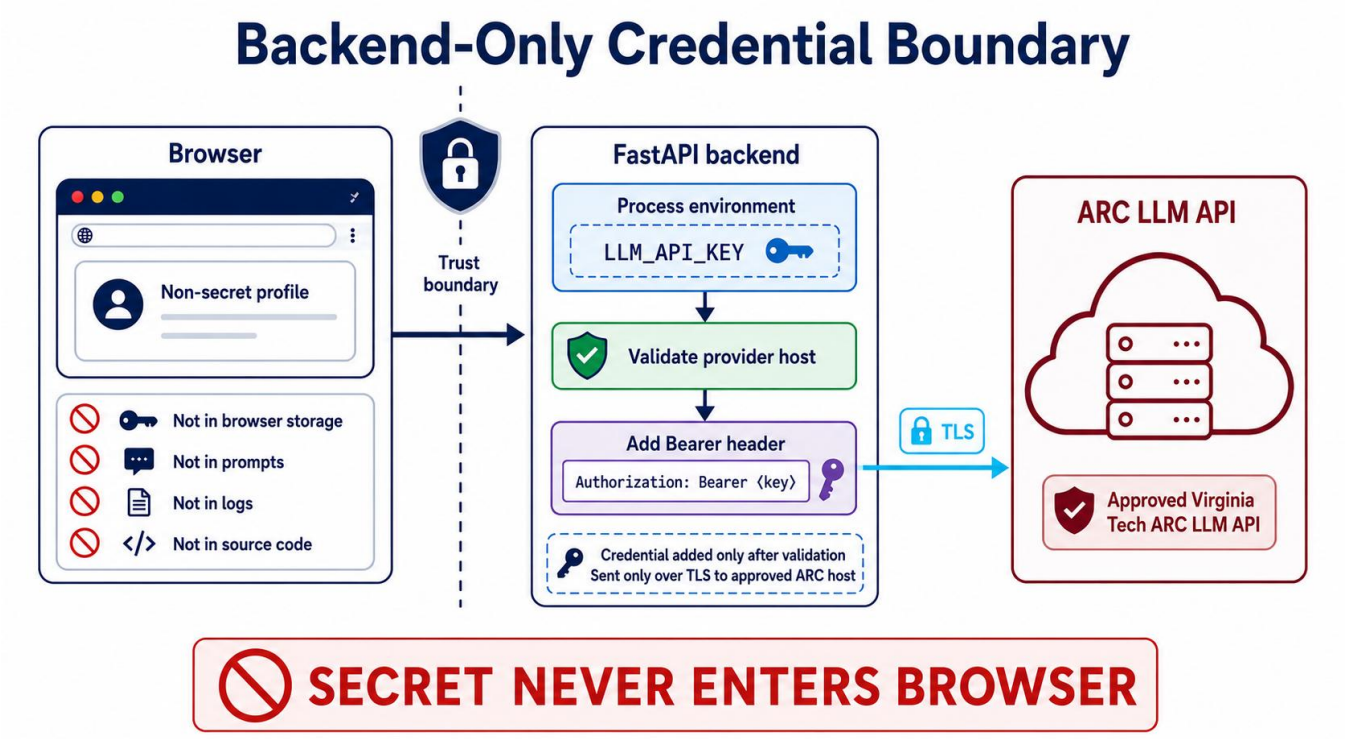

Figure 1: The authoritative credential boundary: the browser holds only a non-secret profile; FastAPI validates the provider host before adding the Bearer header and sending the credential over TLS to the approved ARC endpoint.

Figure 1 places destination validation before authorization. This ordering limits credential-bearing requests to an approved host. TLS protects the credential in transit, while host validation reduces the risk that a browser-supplied or modified base URL could redirect an authorized request to an attacker-controlled endpoint.

3. ARC Provider Integration

ARC provides an OpenAI-compatible base URL at `https://llm-api.arc.vt.edu/api/v1`. Virginia Tech students, faculty, and staff can create a personal key at `https://llm.arc.vt.edu` through **User profile > Settings > Account > API keys**. ARC identifies the key as unique to the user and requires the user to keep it confidential.

The application sends the key through the HTTP `Authorization` header using the Bearer scheme. At the time the source guide was verified, August 27, 2026, the checked-in default profile used `Kimi-K3-thinking-max-legacy-tool-calling` and enabled the model-selected `server:websearch` tool. ARC can add, remove, or rename models. Operators should use the application's model-discovery function and review the ARC documentation before changing the default alias.

The provider connection does not change the model that powers a coding agent. Codex can use ARC only through an authorized project process that inherits `LLM_API_KEY`; project configuration does not grant Codex an independent ARC identity or direct account access.

The project completed its backend credential boundary, provider profiles, model discovery, connectivity testing, and streamed provider calls in G02. Explicit live smoke tests may therefore connect to ARC through the established application path. Routine coding and automated verification should remain credential-free and mocked. G05-S5.1 introduces integration-specific acceptance work for live ARC search behavior; G05-S5.3 covers MCP connectivity and any separate service credentials.

4. Approved Credential-Injection Paths

The application supports two local runtime patterns. Docker Compose reads a protected local file and injects the value into the API container environment. Native development reads the value through a hidden terminal prompt, places it in the launching shell's process environment, and removes it after the development process exits.

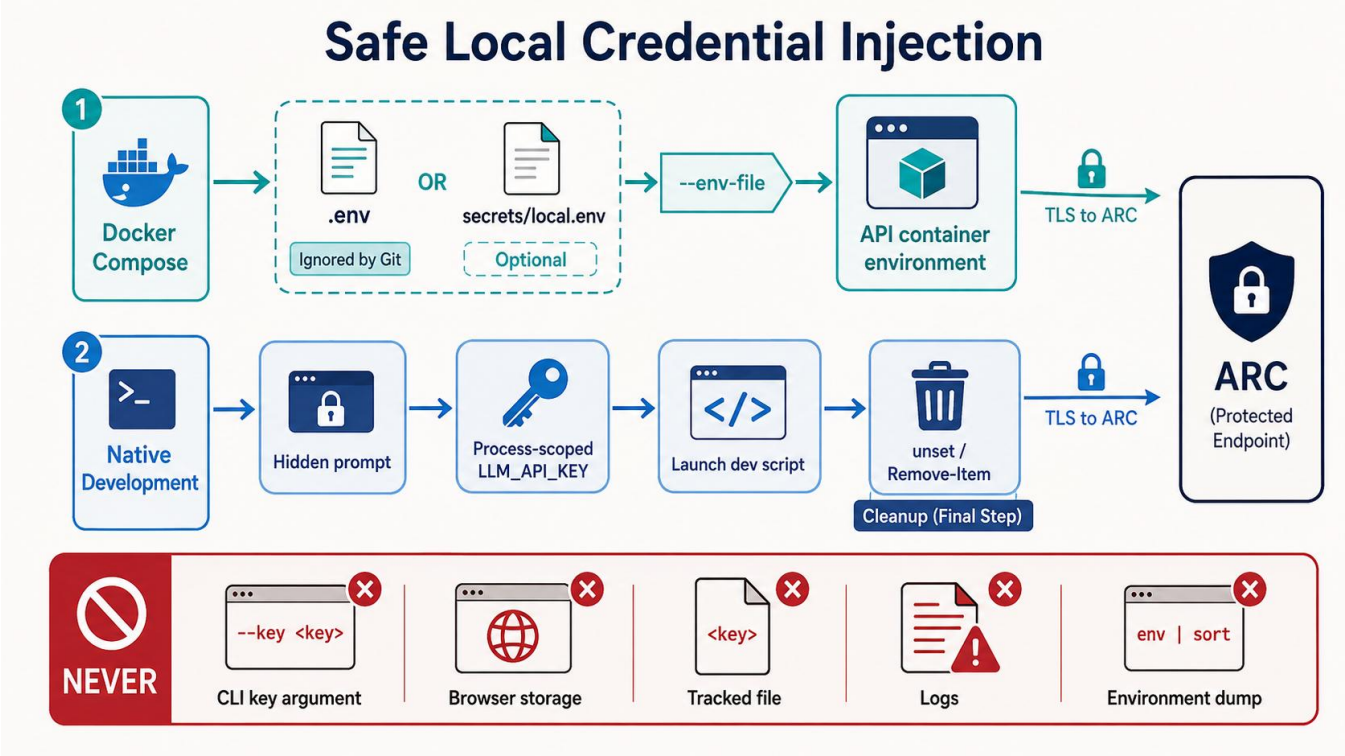

Figure 2: Two approved local credential-injection paths: Docker Compose uses an ignored local environment file, while native development uses a hidden prompt and a process-scoped variable that is removed after use.

4.1 Docker Compose

Docker Compose is the production-shaped local runtime. It automatically reads the root `.env` file for interpolation.

Step 1: Prepare the local file. Copy `.env.example` to `.env`; leave the tracked example unchanged.

Step 2: Set the credential. Set `LLM_API_KEY=<ARC_API_KEY>` in the local `.env` file.

Step 3: Preserve the host restriction. Keep `ALLOWED_PROVIDER_HOSTS` at its checked-in default unless the project intentionally adds another approved provider.

Step 4: Recreate the runtime. Start or recreate the services with `docker compose up --build` so the API container inherits the current value.

Step 5: Test through the application. Open the loopback web URL and run the provider connectivity test. Do not paste the key into the browser.

The repository ignores `.env`, and the Docker build context excludes it. Compose injects the value into the API container environment; this design does not use Docker Secrets. A local user or administrator with Docker inspection privileges may be

able to read container environment variables. The workstation and Docker daemon therefore remain part of the credential's security boundary.

An operator may store an alternative local file at `secrets/local.env` and select it explicitly:

```
docker compose --env-file secrets/local.env up --build
```

Compose does not read `secrets/local.env` unless the operator supplies `--env-file`. The command names the file, not the credential value. Passing the key itself as a command-line argument can expose it through shell history, process inspection, terminal transcripts, and copied support output.

4.2 Windows PowerShell

Native development scripts inherit variables from the shell that launches them; they do not load `.env` or `secrets/local.env`. PowerShell can read the key through a hidden prompt and minimize its lifetime as a managed plain-text string.

```
$SecureArcKey = Read-Host "ARC API key" -AsSecureString
$SecretPointer = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($SecureArcKey)
try {
    $env:LLM_API_KEY = [Runtime.InteropServices.Marshal]::PtrToStringBSTR($SecretPointer)
} finally {
    [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($SecretPointer)
}
./scripts/dev.ps1
Remove-Item Env:LLM_API_KEY -ErrorAction SilentlyContinue
```

The child API process receives a plain environment string because HTTP authentication requires the usable credential. The cleanup command runs after the development script exits. Operators should close the shell when inherited environment state is uncertain.

4.3 POSIX Shell

A POSIX shell can suppress terminal echo while reading the value, export it for the child process, and remove it afterward:

```
IFS= read -r -s LLM_API_KEY
printf '\n'
export LLM_API_KEY
./scripts/dev.sh
unset LLM_API_KEY
```

An inline assignment containing the real key is inappropriate because it can enter shell history, terminal transcripts, process inspection, or troubleshooting output.

5. Protecting Local Secret Material

Ignored files and process environments reduce routine exposure; they do not make a credential inaccessible. Local protection depends on access control, storage choices, disciplined tooling, and a short credential lifetime.

5.1 Key Ownership and File Permissions

Each researcher should use a dedicated personal ARC key. A researcher must not share another user's key or reuse the ARC credential for unrelated systems.

On POSIX systems, restrict an approved secret file to its owner with `chmod 600 <file>`.

On Windows, inspect permissions with `icacls <file>` and remove unintended user or group access through approved local IT practices.

Secret files belong only in documented ignored locations. Avoid synchronized folders, shared drives, email, issue attachments, and unencrypted backups. Clear copied keys from the clipboard after configuration and avoid screen sharing while the ARC key-management page or a secret file is visible.

5.2 Browser and Development Tool Exclusions

Do not store credentials in a VS Code setting, launch configuration, `.envrc`, MCP YAML, browser password field, IndexedDB, localStorage, or frontend build variable. A value prefixed with `VITE_` is compiled for browser use and is never appropriate for a secret.

ARC requires verified TLS. The browser profile's TLS override is a visible diagnostic escape hatch; it is not a credential workaround and should not be used to disable TLS verification for ARC.

5.3 Secondary Disclosure Channels

Crash dumps, debugger captures, environment listings, PowerShell transcripts, CI logs, Docker inspection output, and support bundles may contain environment data or request context. Operators should treat these artifacts as possible credential-bearing material until they verify otherwise.

6. Safe Coding, Testing, and Agent-Assisted Development

The normal unit, component, and browser test suites use mocked providers. These tests should remain deterministic, offline, and independent of a live key. A test that unexpectedly requires `LLM_API_KEY` has crossed the intended integration boundary and should be reviewed.

Live ARC checks must remain explicit and opt-in. A command may inherit `LLM_API_KEY` from the environment, but it must not print the environment, request headers, raw provider bodies, or fully expanded Compose configuration. Logging should record metadata needed for diagnosis without recording authorization material.

Before authorizing a live check, the operator should confirm five conditions:

- The command reads `LLM_API_KEY` from the environment rather than a flag or source file.
- Logging is metadata-only, and error messages sanitize upstream content.
- The test prompt is low cost and contains no sensitive project information.
- The destination resolves to the allowed ARC HTTPS host.
- The operator will remove the process-scoped variable and recreate or stop containers after the check.

Do not run `docker compose config`, `Get-ChildItem Env:`, `env`, `set`, or another diagnostic that dumps full process or container environments into a transcript while a real key is loaded. To inspect Compose structure safely, override `LLM_API_KEY` with an empty value in the isolated verification process.

7. Rotation, Revocation, and Incident Response

Rotate a credential when its intended lifetime ends, a machine or account changes hands, permissions become uncertain, or a possible disclosure occurs. A suspected disclosure in Git, chat, a log, a screenshot, a generated report, or another shared system makes the credential compromised for operational purposes.

Deleting the visible copy does not restore the key's confidentiality.

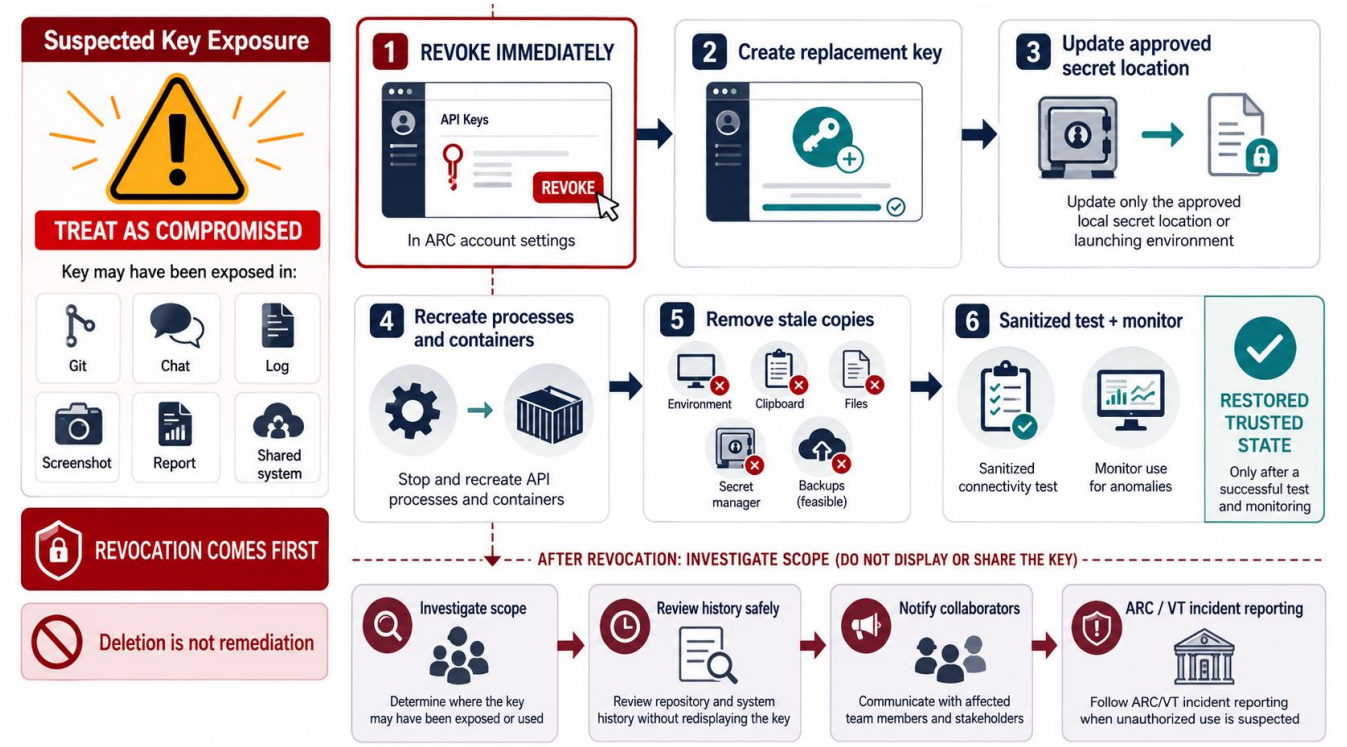

Figure 3: The response to suspected key exposure begins with immediate revocation, followed by replacement, runtime recreation, cleanup, sanitized testing, monitoring, and a parallel investigation of disclosure scope.

The recovery order is deliberate:

Step 1: Revoke. Revoke the affected key through the ARC account API-key settings.

Step 2: Replace. Create a replacement personal key.

Step 3: Update the approved source. Replace the value only in the approved local secret location or launching environment.

Step 4: Recreate runtimes. Stop and recreate each API process and container that inherited the old value.

Step 5: Remove stale copies. Remove stale copies from environment variables, clipboard managers, local secret files, approved secret managers, and feasible backups.

Step 6: Test and monitor. Run a sanitized connectivity test and monitor for unexpected failures or use.

Revocation comes first because an exposed key may remain usable after a user deletes a message, log, screenshot, or repository file. Cleanup reduces continued disclosure; it cannot invalidate the credential at the provider.

After revocation, the project should determine the disclosure scope. Review repository history and artifacts without redisplaying the key. If the key entered Git, use a reviewed history-recovery procedure and notify collaborators who may have fetched the affected objects. Follow ARC and Virginia Tech incident-reporting requirements when unauthorized use is suspected.

8. Repository Safeguards and Their Limits

Repository controls provide defense in depth. Each control addresses a narrow failure mode and leaves other paths open.

Safeguard	Protection provided	Limitation
`.gitignore`	Excludes approved secret files before tracking	Does not protect an already tracked file and does not inspect contents
`.dockerignore`	Excludes local secret material from the Docker build context	Does not hide values injected into a running container environment
`.gitattributes` with `export-ignore`	Excludes named secret paths from generated archives	Does not block `git add`; secret files still must remain untracked
`scripts/security.ps1` and `scripts/security.sh`	Scan repository text for selected credential formats and verify exclusion rules	Pattern coverage is incomplete and cannot prove arbitrary content is safe
`.env.example`	Documents variable names, safe defaults, and blank placeholders	Becomes dangerous if a real credential or operational secret is added

Before committing, run the platform-appropriate security script and inspect staged changes by filename and content. Do not use force-add to bypass an ignore rule for a credential-bearing file. A clean scanner result supports review; it does not replace review.

9. Operational Checklist

Initial Setup

- Create a personal ARC key through the user's ARC account.
- Store it in one approved local source: root `.env`, explicitly selected `secrets/local.env`, or the launching shell environment.
- Confirm file permissions and keep the location outside synchronized or shared storage.
- Retain the checked-in allowed-host default unless another provider has received explicit approval.

Before a Live Check

- Confirm that mocks cover routine automated tests.
- Inspect the command for environment dumps, expanded configuration, request-header logging, or raw response logging.
- Use a low-cost, non-sensitive prompt.
- Confirm the approved ARC HTTPS destination.

After a Live Check

- Remove or unset the process-scoped variable.
- Stop or recreate processes and containers that inherited the credential when appropriate.
- Clear clipboard copies and close shells whose inherited state is uncertain.
- Review sanitized connectivity results without publishing request headers or environment output.

Before a Commit or Shared Support Session

- Run the repository security script.

- Review staged filenames and contents.
- Keep key-management pages, secret files, environment listings, and Docker inspection output out of screenshots and transcripts.
- Verify that `.env.example` contains placeholders and safe defaults only.

10. Discussion

The model favors a practical local-development boundary rather than a claim of perfect local secrecy. The FastAPI process must hold a usable credential long enough to construct an authorized request. Docker administrators, debuggers, crash capture, and process inspection may expose environment values to sufficiently privileged local users. The controls reduce common accidental disclosures and constrain where the key should appear; they do not protect a compromised workstation or a malicious administrator.

Destination validation and credential custody must remain coupled. A backend that keeps the key out of the browser but accepts an arbitrary provider URL can still disclose the credential. A backend that validates the host but logs request headers can expose the same key through operations. The security argument depends on the full chain: approved input source, backend-only custody, pre-authorization host validation, TLS transport, sanitized logging, and prompt revocation after suspected exposure.

Future MCP integration should preserve the same principles while separating credentials by service. Configuration should reference environment-variable names rather than contain literal bearer tokens or stdio credentials. The project should define allowed destinations, sanitized failure behavior, and rotation ownership for each MCP service before enabling it.

11. Conclusion

The project can connect to ARC without placing the researcher's personal credential in the browser or repository. One environment variable supplies the FastAPI process; the backend validates the provider host before adding the Bearer header and sends the authorized request only over TLS. Docker Compose and native development provide documented local injection paths, while mocked providers keep routine testing offline and credential-free.

Operational discipline completes the design. Teams must protect ignored files, avoid environment-dump diagnostics, review staged content, and treat any shared appearance of a key as a compromise. Immediate provider-side revocation, followed by replacement and cleanup, provides the only reliable response to suspected exposure.

Appendix A. Visual Design Evolution

The first credential-boundary illustration correctly separated the browser from the backend and placed host validation before the Bearer header. Its small backend note stated that the credential existed only in backend process memory and never left. That wording was imprecise because the backend must transmit the credential to the approved provider inside the TLS-protected authorization header. Figure 1 is the authoritative version; it replaces the note with language that matches the implemented flow.

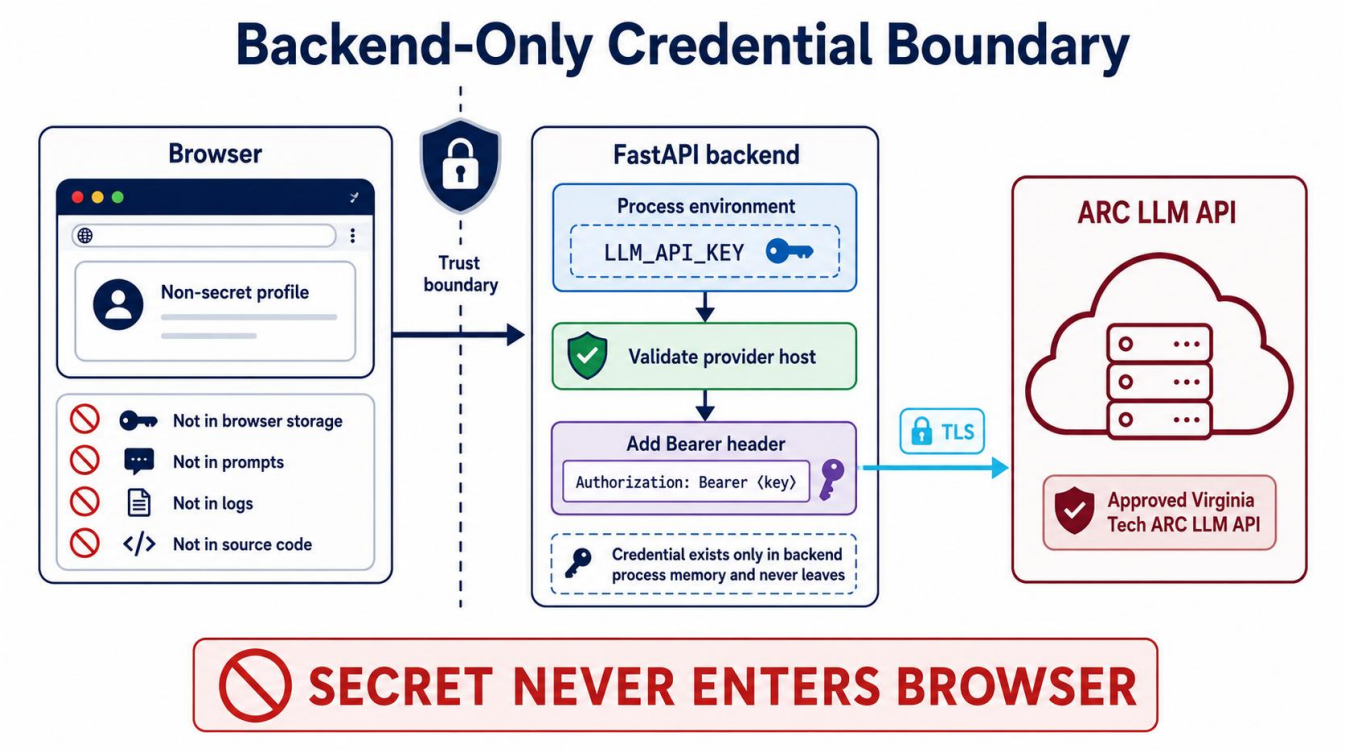

Figure 4: Initial credential-boundary concept retained for design traceability. The bottom note in the FastAPI panel is superseded by Figure 1 and must not be used as current operational guidance.

The correction illustrates a useful documentation rule: security diagrams must distinguish browser exclusion from provider transmission. The key never enters the browser, but it does leave backend memory as part of the authorized TLS request to the validated ARC host.

References

[1] Virginia Tech Advanced Research Computing. "LLM API Service." ARC Documentation. https://www.docs.arc.vt.edu/ai/011*llm*api*arc*vt_edu.html. Configuration details in the source guide were last verified August 27, 2026.

[2] Project documentation. "Secrets and Keys Management." Internal Markdown guide, August 27, 2026.